

08. DDPM & Stable Diffusion

Overview

Denoising Diffusion Probabilistic Models (DDPM) implement discrete-time reverse steps with learned noise prediction. We cover the simplified training loss, sampler schedules, classifier-free guidance, and how Stable Diffusion moves diffusion to a VAE latent space for efficiency.

Lab: Lab 08.

Prior modules: Module 07.

Contents

1	DDPM Formulation	2
2	Simplified Training Loss	2
3	Sampling Algorithm	2
4	U-Net Denoiser Backbone	2
5	Stable Diffusion: Latent Space	2
6	Classifier-Free Guidance	2
7	Lab	3
8	Noise Schedules in Practice	3
8.1	EMA Weights	3
9	Extended Intuition	3
9.1	Reverse Process and Variance	4
10	Numerical Walkthrough	4
11	PyTorch Implementation Notes	4
12	Local GPU Constraints	5
13	Debugging Checklist	5
14	Practice Problems	5

Module track

This module builds on **Module 07**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

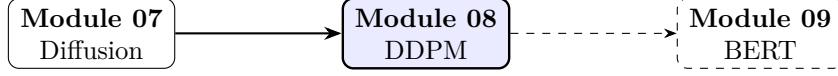


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 DDPM Formulation

Denoising Diffusion Probabilistic Models [1] simplify the reverse variance and reparameterize mean as noise prediction. Builds on forward process **Module 07** (forward diffusion).

2 Simplified Training Loss

Ho et al. show a weighted denoising score matching objective:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2 \right], \quad (1)$$

where $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, $t \sim \text{Uniform}(1, T)$.

Note. Predict noise ϵ rather than $\mathbf{x}_0$ directly, numerically stable and equivalent under reparameterization.

3 Sampling Algorithm

Starting from $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$, iterate for $t = T, \dots, 1$:

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right), \quad (2)$$

$$\mathbf{x}_{t-1} = \mu_{\theta} + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}) \text{ if } t > 1. \quad (3)$$

4 U-Net Denoiser Backbone

ϵ_{θ} is a U-Net with timestep embedding (sinusoidal, as in **Module 04** (transformer block)). Skip connections preserve spatial detail, architecture shared with segmentation in **Module 14** (U-Net architecture).

5 Stable Diffusion: Latent Space

Stable Diffusion applies diffusion in VAE latent $\mathbf{z}$ rather than pixel space, reducing compute:

$$\mathcal{L} = \mathbb{E} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{z}_t, t, \mathbf{c})\|^2 \right], \quad (4)$$

where $\mathbf{c}$ is text conditioning via cross-attention (transformer blocks from **Module 04**).

6 Classifier-Free Guidance

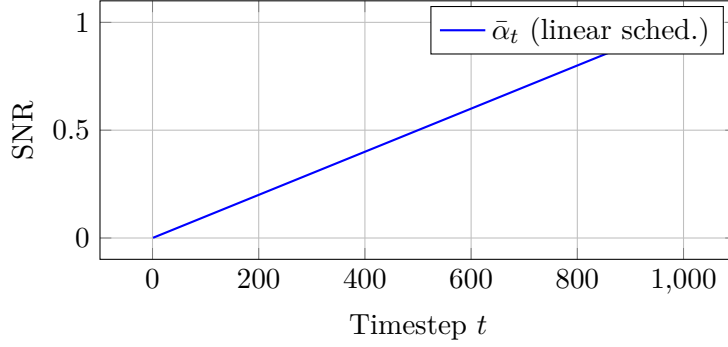
Interpolates conditional and unconditional noise predictions:

$$\tilde{\epsilon}_{\theta} = \epsilon_{\theta}(\mathbf{z}_t, t, \emptyset) + w (\epsilon_{\theta}(\mathbf{z}_t, t, \mathbf{c}) - \epsilon_{\theta}(\mathbf{z}_t, t, \emptyset)). \quad (5)$$

$w > 1$ sharpens samples at cost of diversity.

Table 1: Pixel vs. latent diffusion.

Aspect	Pixel DDPM	Latent (SD)
Resolution	Up to 256^2	512^2 – 1024^2
VRAM	High	Moderate
Text conditioning	Add-on	CLIP + cross-attn

Figure 2: Signal-to-noise ratio decreases as t increases.

7 Lab

Lab 08 trains a DDPM on MNIST using (1); compare sample quality to GANs in **Module 06** (DCGAN guidelines).

8 Noise Schedules in Practice

Linear β_t schedule from **Module 07** (noise schedule) works for DDPM; latent models often use cosine. Classifier guidance adds $\nabla_{\mathbf{x}_t} \log p(c | \mathbf{x}_t)$ to the score during sampling.

8.1 EMA Weights

Maintain exponential moving average $\theta_{\text{EMA}} \leftarrow 0.999 \theta_{\text{EMA}} + 0.001 \theta$ for stabler samples.

9 Extended Intuition

DDPM implements discrete reverse steps that invert the forward noising process learned during training. At inference you start from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iterate $t = T, \dots, 1$ using the network’s noise predictions. Each reverse step adds a small amount of Gaussian noise (except often at $t = 1$) to match the learned transition kernels.

The simplified training objective from Ho et al. is mean squared error on ϵ :

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2 \right]. \quad (6)$$

Sampling uses ϵ_{θ} to estimate denoised direction; DDIM and other samplers reduce step count for deployment.

Stable Diffusion runs diffusion in a VAE latent space ($\mathbf{z} \in \mathbb{R}^{4 \times 64 \times 64}$ for 512×512 images) so attention cost drops. Classifier-free guidance interpolates conditional and unconditional noise predictions at sample time for sharper text-conditioned images. Fine-tuning Stable Diffusion on a 1650 is usually impractical for full weights; LoRA adapters from **Module 11** are the practical path.

9.1 Reverse Process and Variance

Ho et al. fix reverse variance to $\tilde{\beta}_t = \frac{1-\bar{\alpha}_t-1}{1-\bar{\alpha}_t}\beta_t$ (posterior variance under Gaussian assumptions). Learned variance (Improved DDPM) adds channels to the network output; helpful at high resolution but more tuning on MNIST-scale labs. DDIM treats reverse steps as non-Markovian with noise scale η ; $\eta = 0$ gives deterministic sampling, $\eta = 1$ matches DDPM stochasticity. Respacing: train with $T = 1000$ but sample with 50 evenly spaced timesteps by jumping $\bar{\alpha}_t$ indices; quality often holds on simple datasets. Probability flow ODE (advanced) treats reverse diffusion as an ODE with $\eta = 0$ DDIM; useful mental model when reducing steps below 50 on MNIST demos. Improved DDPM learns diagonal variance in $\mathbf{x}_0$ prediction; skip unless basic MNIST sampling already looks sharp at 1000 steps.

10 Numerical Walkthrough

MNIST DDPM: 28×28 , $T = 1000$, U-Net base channels 64, $B = 64$, $\text{lr} = 2 \times 10^{-4}$.

Training step: sample batch images $\mathbf{x}_0$, random $t \in \{0, \dots, 999\}$, MSE on predicted vs true noise. Typical initial loss ≈ 1.0 (unit variance noise in 784 dims per pixel channel averaged) dropping toward 0.05–0.15 when converging.

Reverse step (conceptual, DDPM):

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}. \quad (7)$$

Full sampling = 1000 network evaluations per image (slow on 1650: minutes per sample without step reduction).

DDIM with 50 steps: subsample timesteps, deterministic update ($\eta = 0$), quality often acceptable on MNIST for labs. Latent SD: VAE encodes $512 \times 512 \rightarrow 64 \times 64 \times 4$; diffusion U-Net operates on 64px grid with cross-attention to text embeddings.

Classifier-free guidance at inference: $\tilde{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset) + w(\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset))$ with guidance weight $w \approx 7.5$ for SD. Training for guidance: randomly drop text condition to empty prompt 10 to 20% of steps so the model learns unconditional denoising alongside conditional. v-prediction parameterization (advanced) targets velocity $\mathbf{v} = \sqrt{\bar{\alpha}_t} \boldsymbol{\epsilon} - \sqrt{1 - \bar{\alpha}_t} \mathbf{x}_0$; some schedulers prefer it at low noise levels.

Sampling step count dominates wall time: 1000 U-Net forwards at 28×28 on a 1650 may take 2 to 5 minutes per image batch; profile one reverse loop before training 200k steps. Classifier guidance (deprecated in text-to-image) adds $\nabla_{\mathbf{x}_t} \log p(c | \mathbf{x}_t)$ to the score; classifier-free guidance replaced it by training without labels intermittently. Latent diffusion trades spatial resolution for channel depth; VAE scale factor 0.18215 appears in SD pipelines and must match when decoding samples. PLMS and DPM-Solver reduce steps below DDIM on some checkpoints; stick to DDIM in the base lab before chasing sampler variants.

11 PyTorch Implementation Notes

- U-Net: downsampling conv blocks with skip connections, timestep injection via `nn.Embedding` + linear projection into feature maps.
- `diffusers` library: `DDPMPipeline`, `UNet2DModel`, `DDPMScheduler` for reference implementations.
- Sampling loop: `model.eval()`, `torch.no_grad()`, iterate scheduler timesteps in reverse order.
- For MNIST lab, 1-channel U-Net; Stable Diffusion checkpoints expect 4-channel latent input and CLIP text encoder outside U-Net.

- EMA weights (exponential moving average of model params) often improve sample quality; update every step with decay 0.9999.

```
for t in scheduler.timesteps:
    eps = model(x, t).sample
    x = scheduler.step(eps, t, x).prev_sample
```

12 Local GPU Constraints

MNIST DDPM U-Net, $B = 64$, trains on 4 GB with fp32; sampling 1000 steps is the bottleneck, not training memory. Use DDIM 50 to 100 steps for demo grids during development.

Full Stable Diffusion inference (fp16) needs 6 to 8 GB+; on 1650 use `enable_attention_slicing`, `enable_vae_slicing`, or CPU offload from `diffusers`. Dream-Booth/LoRA fine-tune rank-4 adapters on small image sets at 512 px with $B = 1$, gradient accumulation 8+.

13 Debugging Checklist

- Gray mush samples: insufficient training steps or wrong image scaling ($[0, 1]$ vs $[-1, 1]$).
- NaN during sampling: fp16 overflow; force fp32 in sampler or clamp $\mathbf{x}_t$.
- Periodic vertical stripes: U-Net alignment bug in skip connections (shape mismatch padded incorrectly).
- SD out-of-memory: reduce resolution to 384, use xFormers/memory efficient attention if available.
- Text conditioning ignored: classifier-free guidance scale 1.0; try 7.5 at inference for SD pipelines.
- Sample grid identical across runs: forgot to fix RNG seed for initial $\mathbf{x}_T$; each sample should start from independent noise.

14 Practice Problems

Problem 1. Compare sample quality and wall time for 1000-step DDPM vs 50-step DDIM on the same checkpoint.

Problem 2. Implement EMA weight tracking and generate samples from EMA vs raw weights.

Problem 3. Estimate VRAM for U-Net forward at 28×28 vs 64×64 with base channels 128.

Problem 4. Run pretrained Stable Diffusion with guidance scale 1, 7.5, 15 and note artifact tradeoffs.

Problem 5. Implement respaced sampling with 50 timesteps from a 1000-step trained model. Compare FID or visual sharpness to full 1000-step chain.

Problem 6. Log MSE per timestep bucket ($t \in [0, 333), [333, 666), [666, 999]$). Which range dominates error at epoch 10 vs epoch 50?

U-Net skip connections must match spatial dims exactly; off-by-one upsample padding causes crop artifacts in generated MNIST digits. Save intermediate $\mathbf{x}_t$ grids every 100 reverse steps during sampling; stuck noise patterns often mean wrong σ_t or scheduler index. Compare EMA vs raw weights on the same noise seed before declaring sampling broken; EMA often fixes speckle without retraining. When running SD demos on 1650, enable sequential CPU offload before lowering resolution below 384px. DDIM η interpolates stochasticity: sweep $\eta \in \{0, 0.5, 1\}$ on

MNIST samples after training converges. Profile one full reverse loop before scaling training; sampling not forward pass usually limits DDPM lab turnaround.

Train DDPM in **Lab 08**; compare GAN samples from **Module 06**. Adapter tuning connects to **Module 11**.

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- [2] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021.
- [3] Robin Rombach, Andreas Blattmann, Dominik Lorenz, et al. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- [4] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015.
- [5] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, et al. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.